

MODULE 19

Integration Testing and UI Test Automation

Learn how to validate your application as a whole and automate real user interactions with confidence.





MODULE 19 OVERVIEW

Validate your application as a whole and automate real user interactions with Selenium.

- Unit tests verify pieces in isolation, but real confidence comes from testing components working together and validating the UI a user actually sees. This module covers integration testing, Spring Test, and Selenium-based UI automation.

DURATION & LAB



1 Hour Theory

Integration testing, Spring Test, Selenium.



2.5 Hours Lab

Integration and UI Testing with Selenium.



Scenario

Regression-test a CRM app end-to-end.

INTEGRATION TESTING

Validate components working together

UI TEST AUTOMATION

Automate user workflows with Selenium

THE PAYOFF

Quality, reliability, faster delivery

KEY TAKEAWAY Good automation is reliable, maintainable, and provides confidence in every release.

WHY INTEGRATION TESTING MATTERS

Unit tests pass, but the application can still fail when components are wired together.



Catch Integration Bugs

- Find issues that only appear when components interact.



Validate Real Wiring

- Verify controllers, services, and repositories work together.



Test Real Protocols

- Exercise HTTP, database, and messaging boundaries.



Build Confidence

- Confirm real components behave correctly together.



Catch Config Issues

- Find misconfigured beans, profiles, and properties.



Ship Faster

- Catch issues early and release with confidence.

PRECISION: Integration tests verify that selected components, infrastructure, and application boundaries work correctly together. End-to-end tests validate complete user journeys through a running system.

KEY TAKEAWAY Integration tests catch the bugs that unit tests can't — the ones that only appear when components work together.

UNIT TESTING VS INTEGRATION TESTING

Both are essential — they test different things at different levels.

ASPECT	UNIT TESTING	INTEGRATION TESTING
Scope	A single class or method in isolation	Multiple components working together
Dependencies	Controlled to isolate behavior (mocks/stubs)	Real implementations for the boundary; unrelated externals replaced
Speed	Very fast	Slower than unit tests
Purpose	Verify business logic correctness	Verify components integrate correctly
Tools	JUnit 5, Mockito	Spring Test, @SpringBootTest, Testcontainers

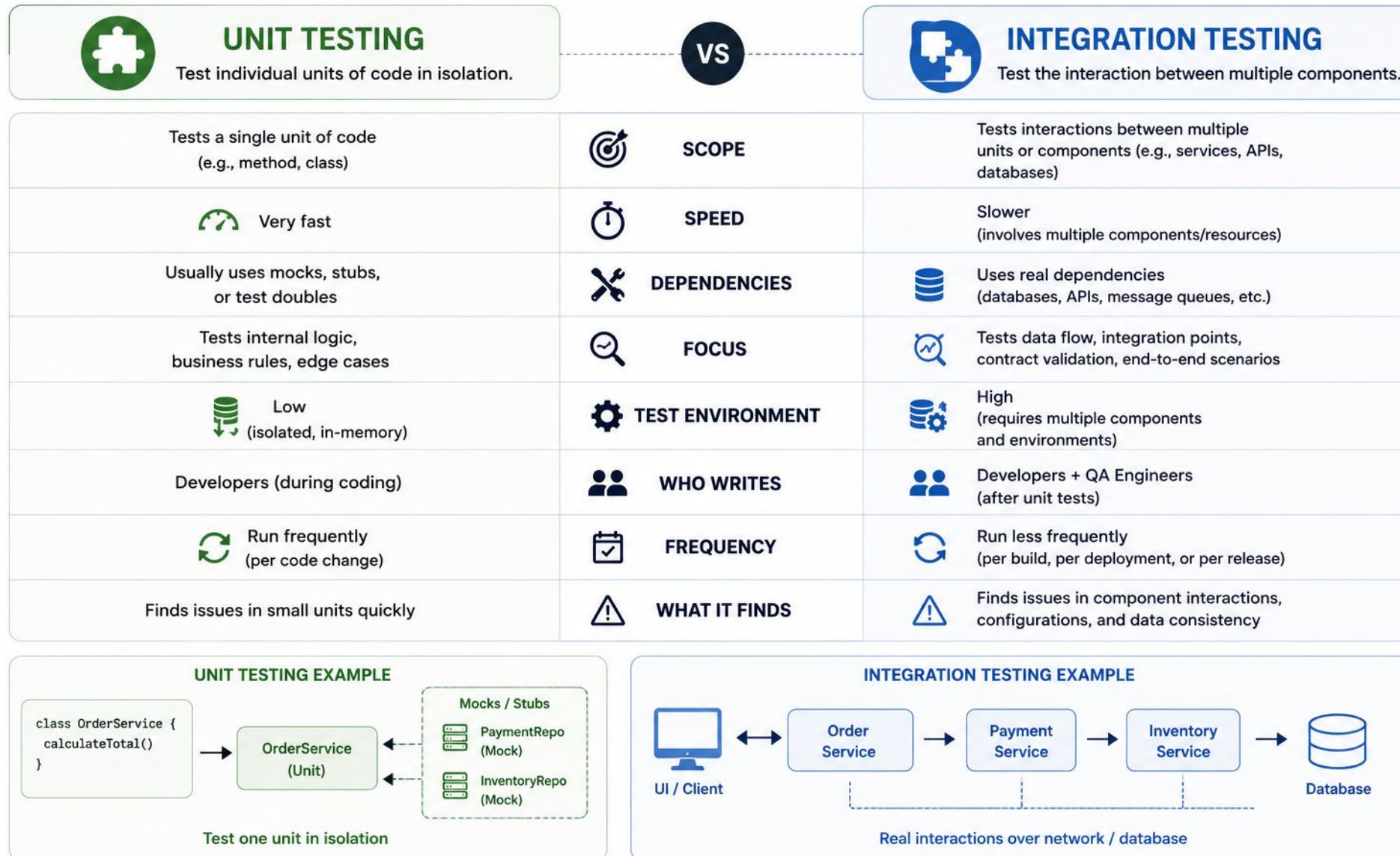
PRECISION: Unit tests control dependencies to isolate behavior. Integration tests use real implementations for the selected integration boundary while replacing unrelated external systems where necessary.

“Unit tests build confidence in the code you write. Integration tests build confidence in the system you deliver.”

KEY TAKEAWAY Use both: fast unit tests for logic, integration tests for confidence that the pieces fit together.

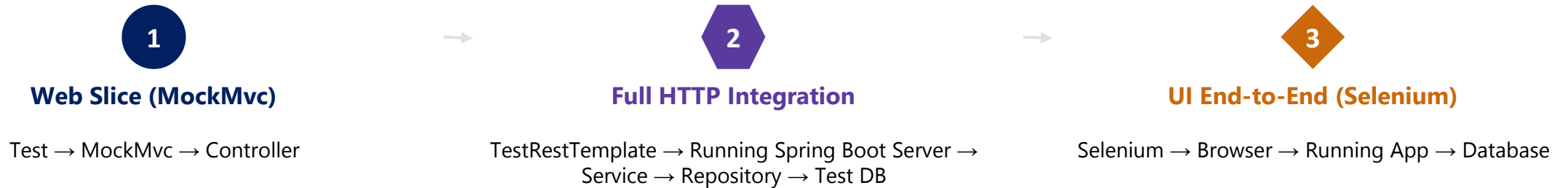
UNIT TESTING vs INTEGRATION TESTING

Both are essential and complementary in a robust testing strategy.



INTEGRATION TESTING ARCHITECTURE

Different test boundaries use different runtime models, from a web-layer slice to a full browser end-to-end run.



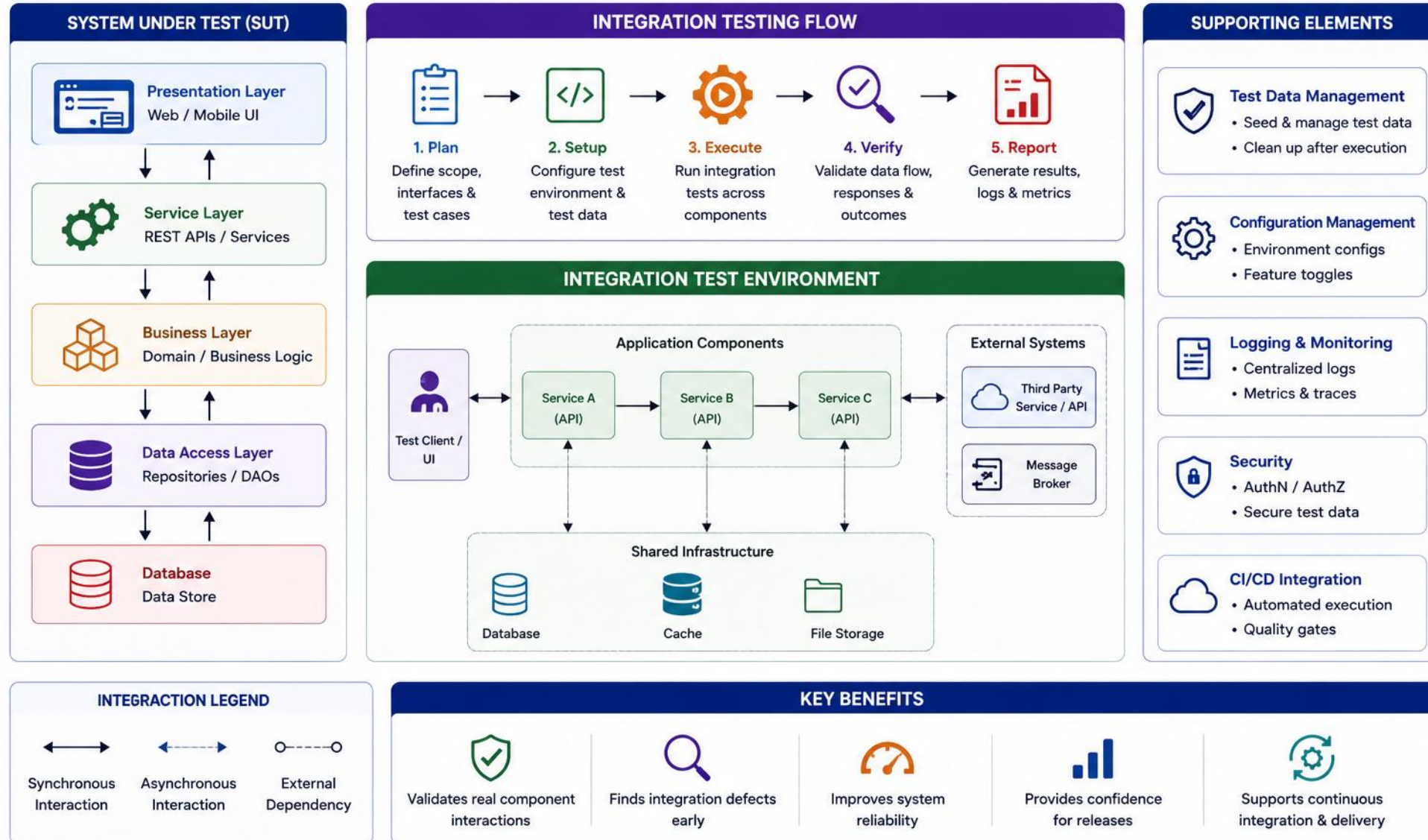
KEY POINTS

- Spring-based integration tests may load a full or sliced application context depending on the boundary being tested.
- Use an in-memory database for fast basic tests only when compatibility is sufficient. Use Testcontainers with the production database engine when database-specific behavior matters.
- Use a test profile with an isolated database so integration tests don't affect production data or other test runs.
- Applications are integrated ecosystems: data flows from the UI through business logic to the database. Integration tests catch interface mismatches and data issues before they reach production.

KEY TAKEAWAY Integration tests trade some speed for confidence that real components genuinely work together.

INTEGRATION TESTING ARCHITECTURE

Validates interactions and data flow between integrated components in an environment close to production.

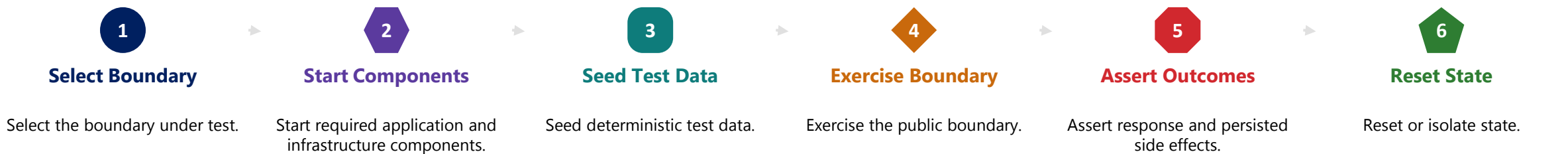


INTEGRATION TESTING ARCHITECTURE (CONT.)

A clear 3-level table for test scope, and the technical workflow for running integration tests.

TEST LEVEL	PURPOSE
Component integration	Multiple application components in one process
Infrastructure integration	Database, messaging, filesystem, HTTP adapter
End-to-end/system	Full running system through external interfaces

INTEGRATION TESTING FLOW (TECHNICAL)



KEY TAKEAWAY Component, infrastructure, and end-to-end/system tests each validate a different boundary — select the boundary, start the system, seed data, exercise it, assert, and reset.



SPRING TEST FRAMEWORK

Three clear groups: slice tests, full application tests, and supporting configuration.

ANNOTATION	PURPOSE	TYPICAL USE
SLICE TESTS — test one layer in a partial context		
@WebMvcTest	Spring MVC test slice: request mapping, serialization, validation, web-layer behavior.	Testing controllers in isolation
@DataJpaTest	JPA/repository layer; may substitute an embedded DB unless configured otherwise.	Testing repository queries
FULL APPLICATION TESTS — the entire context, running or mocked		
@SpringBootTest	Loads a full or sliced context; 4 web-environment options (next slide).	Full integration tests
SUPPORTING CONFIGURATION — shape the context, not the boundary		
@ActiveProfiles	Activates specific Spring profiles for testing.	Selecting test vs. prod config
@TestPropertySource	Overrides properties for the test.	Pointing to a test database
@AutoConfigureMockMvc	Auto-configures MockMvc for web layer tests.	REST endpoints, no running server
@Transactional	Rolls back test data, but may hide commit-time behavior (see next slide).	Isolating test data between runs
@MockBean	Replaces a context collaborator with a test double (see next slide).	Excluding one collaborator's real behavior

SEE NEXT SLIDE: Annotation precision notes and the @SpringBootTest RANDOM_PORT example used in the Lab 19 HTTP integration tests.

KEY TAKEAWAY Choose the narrowest Spring Test annotation that gives you confidence — full context tests are powerful but slower.

SPRING TEST FRAMEWORK (CONT.)

Annotation precision notes, and the @SpringBootTest web-environment options behind the lab's HTTP tests.

@MockBean Replace selected application-context collaborators with test doubles only when the slice under test should exclude their real behavior. Do not confuse this with a pure unit test. (Spring Boot version affects the exact API — align examples accordingly.)

@WebMvcTest Loads the Spring MVC test slice for controller request mapping, serialization, validation, and web-layer behavior, while other collaborators are usually replaced.

@DataJpaTest Configure the test explicitly when the goal is to test against the production database engine, especially with Testcontainers.

@Transactional Transactional rollback helps isolate test data but may hide commit-time behavior. Use explicit commit or non-transactional tests when the production behavior depends on transaction completion.

@SpringBootTest: 4 WEB-ENVIRONMENT OPTIONS

- **MOCK** (default) — A mock servlet environment — no real HTTP port is opened.
- **RANDOM_PORT** — Starts a real server on a random free port — used by the lab's CustomerApiIT.
- **DEFINED_PORT** — Starts a real server on a configured, fixed port.
- **NONE** — No web environment or server is started at all.

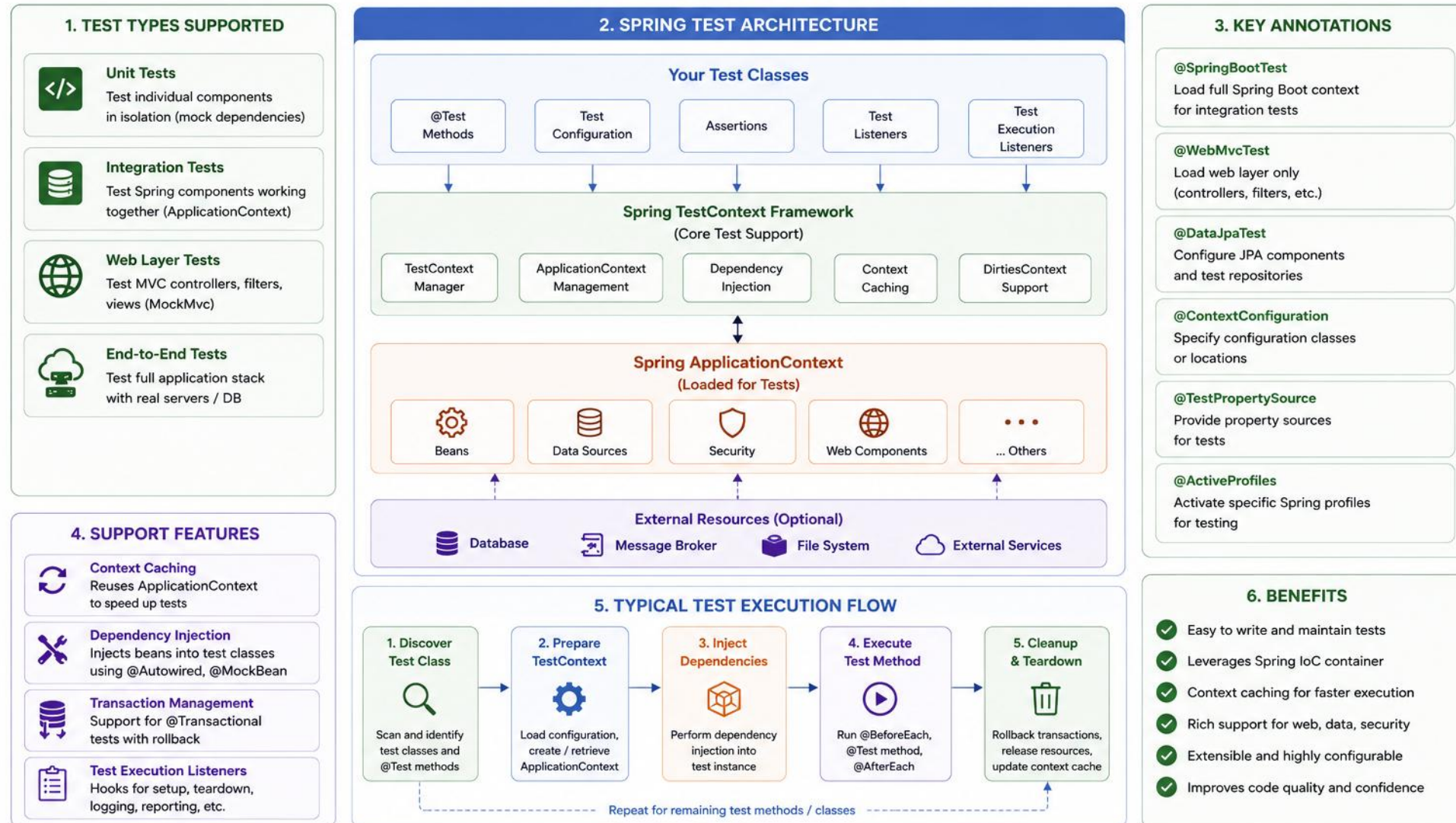
RANDOM_PORT example — used in Lab 19's CustomerApiIT

```
@SpringBootTest(  
    webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT  
)
```

KEY TAKEAWAY Read the fine print: @DataJpaTest's embedded database and @Transactional's rollback can hide real production behavior unless you configure them deliberately.

SPRING TEST FRAMEWORK

Comprehensive testing support for Spring applications.



Spring Test Framework provides a unified and flexible foundation for testing Spring-based applications at every layer.

END-TO-END TESTING CONCEPTS

End-to-end (E2E) tests validate a complete user journey through the real, running application.

E2E sits atop the test pyramid — few, slow, high-value tests above many fast unit tests and some integration tests.



Full Stack

- Exercises the entire application through external interfaces.



Real User Journeys

- Follows what a real user actually does, end to end.



Slowest, Fewest

- Few tests, but each covers a long real path.



Strong Confidence

- Provides strong confidence for critical journeys, but cannot replace lower-level tests.



Highest Cost

- Most expensive to write, run, and maintain.



Complement Other Tests

- Layer on top of unit and integration coverage, not instead of it.

PRECISION: E2E tests provide strong confidence that selected critical journeys work through the deployed stack, but they cannot replace lower-level tests.

System under test

Real

Controlled infrastructure

Real test instance/container

Third-party systems

Sandbox / stub / contract adapter

Production systems

NEVER called from tests

KEY TAKEAWAY E2E tests give strong confidence at the highest cost — use them sparingly, for critical user journeys, and keep lower-level tests too.

END-TO-END TESTING CONCEPTS

Validate the complete user journey across the entire application stack in a production-like environment.

1. WHAT IS END-TO-END TESTING?



End-to-End (E2E) testing validates **complete business workflows** from the user interface through all layers to external systems and back, just like real users.

2. E2E TEST SCOPE



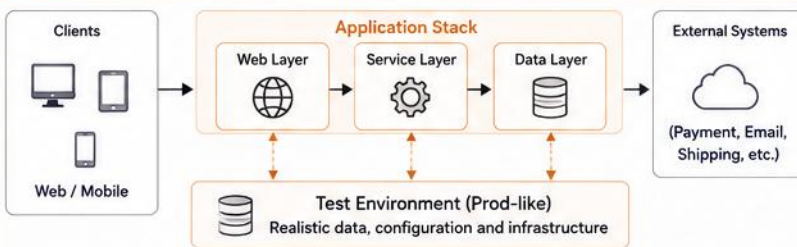
3. KEY CHARACTERISTICS

- ✓ Tests complete user workflows
- ✓ Validates integrated components
- ✓ Uses real environment & data (or close to real)
- ✓ Black-box perspective (from user view)
- ✓ Slower and more expensive than unit/integration tests

4. EXAMPLE END-TO-END FLOW (ONLINE ORDER PLACEMENT)



5. E2E TEST ARCHITECTURE



6. BENEFITS

- ✓ Validates complete business workflows
- ✓ Detects issues in component integration
- ✓ Ensures data consistency across systems
- ✓ Improves user experience and reliability
- ✓ Builds confidence before production release

7. BEST PRACTICES

- 📌 Design tests around critical user journeys
- 📌 Use stable and maintainable test data
- 📌 Run in environments close to production
- 📌 Follow Page Object Model / modular design
- 📌 Execute tests as part of CI/CD pipeline
- 📌 Monitor and report results with logs/screenshots

8. WHEN TO USE E2E TESTING?



Before major releases



To validate critical business processes



When multiple components work together



To ensure production readiness



For user acceptance validation

KEY TAKEAWAY



E2E testing ensures the application works as a whole from the end user's perspective, delivering confidence in real-world scenarios.

SELENIUM ECOSYSTEM

Selenium WebDriver is a widely adopted standard for cross-browser automation and remains common in Java enterprise testing.



Selenium IDE (Prototyping)

- Can help prototype or inspect flows, but production suites should use maintainable coded tests, stable locators, assertions, and page abstractions.



Selenium WebDriver

- Core component for writing advanced automated tests with code. This course standardizes on WebDriverManager for driver setup.



Selenium Grid

- Runs tests in parallel across multiple machines, browsers, and platforms.

HISTORICAL NOTE: Selenium RC was the earlier remote-control tool. It has been fully replaced by WebDriver.

COMMON USE CASES

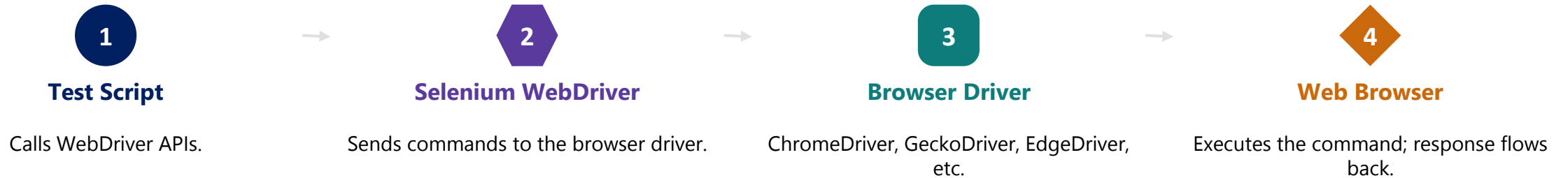
- Functional and cross-browser testing of web applications.
- Regression testing of critical workflows, and data-driven, repeated test execution.
- Continuous testing in CI/CD pipelines, from UI to backend.

TOOLING CHOICE: This course standardizes on WebDriverManager, matching the lab, rather than teaching both driver-management approaches side by side. Use Selenium Manager only if your Selenium version supports the environment reliably; use WebDriverManager if the project intentionally standardizes on it.

KEY TAKEAWAY Selenium WebDriver is the modern standard: IDE helps prototype and inspect, Grid enables parallel execution, and RC is historical.

SELENIUM ARCHITECTURE

Selenium follows a client-server architecture: test scripts send commands to WebDriver, which controls the browser.



KEY BENEFITS

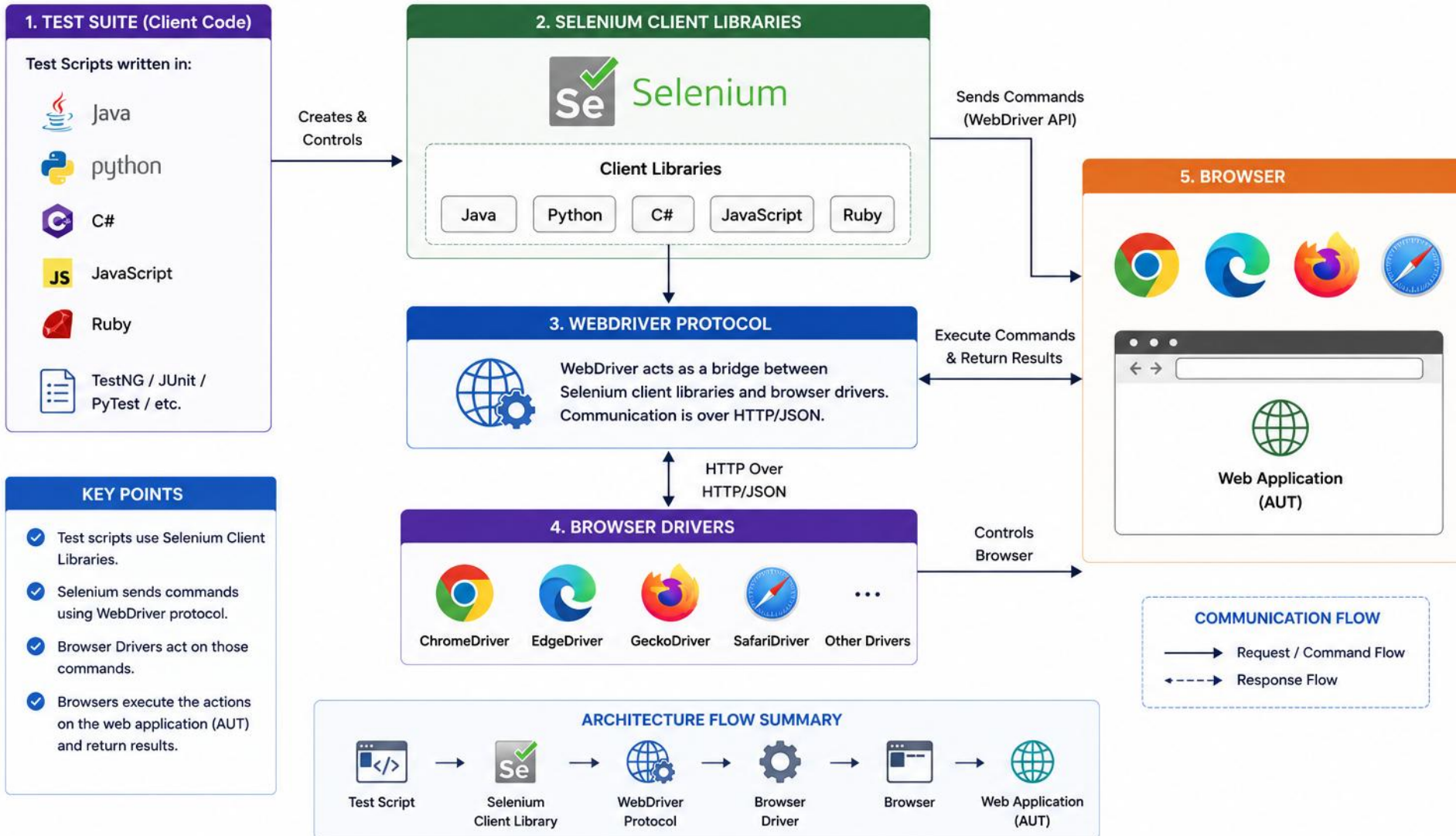
- Language independent, supports multiple browsers, enables parallel execution, and integrates with CI/CD tools.

Test code uses the Selenium WebDriver API. Selenium communicates with the browser's WebDriver implementation using the W3C WebDriver protocol.

KEY TAKEAWAY The WebDriver protocol is the bridge between your test code and the real, running browser.

SELENIUM ARCHITECTURE

Selenium automates browsers. Tests are written in a language, executed via WebDriver, which communicates with the browser to perform actions and validate results.



WEBDRIVER FUNDAMENTALS

WebDriver is the API you use to control the browser: navigate, find elements, and perform actions.

EXAMPLE

```
@BeforeEach
void startBrowser() {
    driver = new ChromeDriver(options);
}
@AfterEach
void stopBrowser() {
    if (driver != null) {
        driver.quit();
    }
}
driver.get("https://demo.ecommerce.test");
WebElement search = driver
    .findElement(By.id("search-box"));
search.sendKeys("laptop");
driver.findElement(By.id("search-button")).click();
```

CORE CAPABILITIES

- Navigate to URLs and manage browser windows.
- Find elements using locators (By.id, By.cssSelector, etc.).
- Perform actions — click, type, submit, hover.
- Wait for elements and assert on page state.

BROWSER SETUP CHECKLIST

- Headless configuration for CI runs
- Explicit window size
- Download directory, if the workflow needs one
- An explicit wait duration (not implicit waits)
- A fresh browser per test, or a carefully managed lifecycle
- No shared cookies/session unless intentional

KEY TAKEAWAY WebDriver is your programmatic remote control for the browser — every UI test is built on these fundamentals.

WEBDRIVER FUNDAMENTALS — EXPLICIT WAITS

Explicit waits are the fundamental missing piece: wait for a real condition, not a fixed sleep.

EXAMPLE

```
WebDriverWait wait =  
    new WebDriverWait(driver, Duration.ofSeconds(10));  
WebElement saveButton =  
    wait.until(  
        ExpectedConditions.elementToBeClickable(  
            By.cssSelector("[data-testid='save-customer']")  
        )  
    );  
saveButton.click();
```

TYPES OF EXPLICIT WAIT CONDITIONS

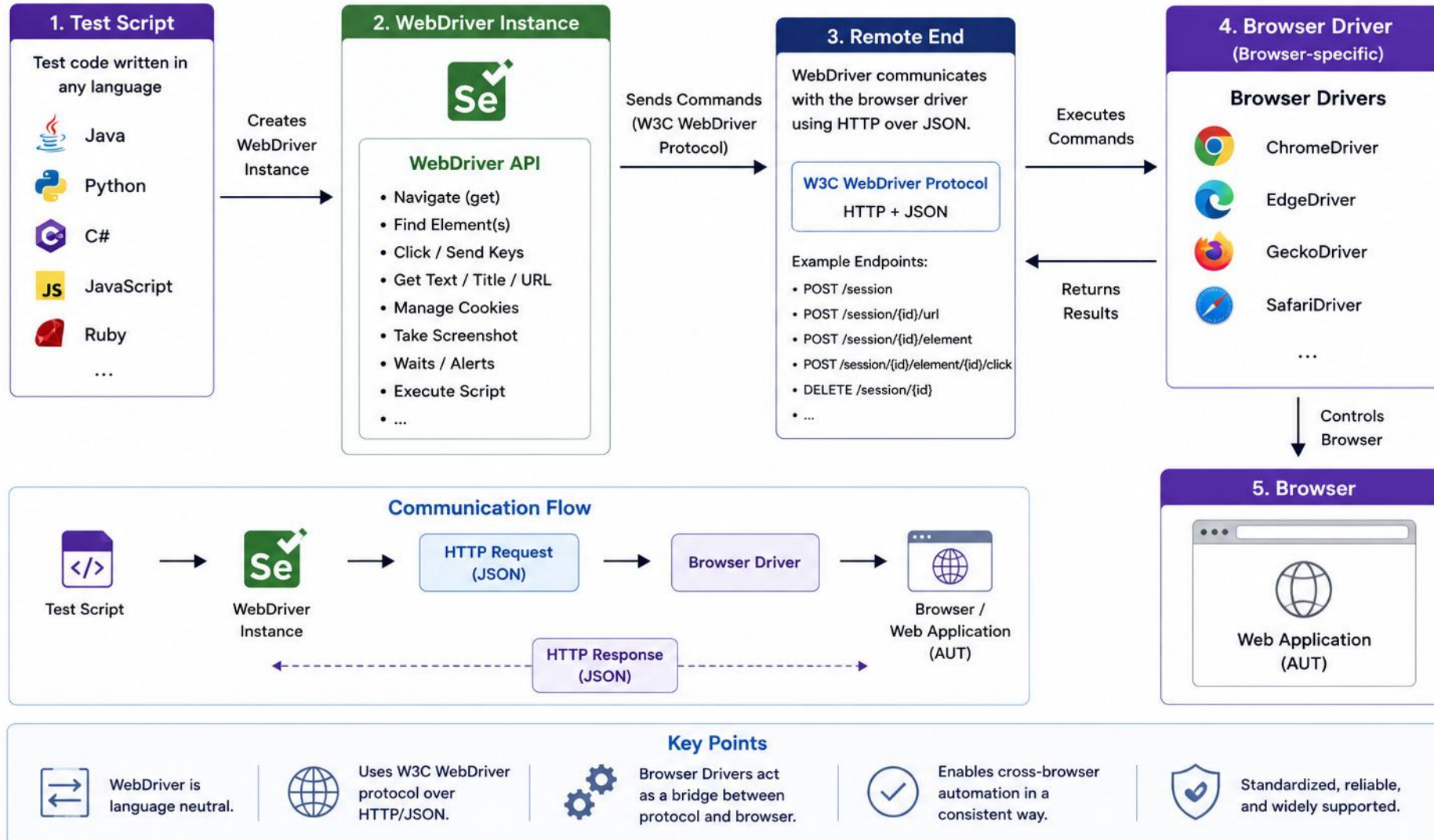
- Presence — the element exists in the DOM.
- Visibility — the element is displayed on the page.
- Clickability — the element is visible and enabled.
- URL / title change — navigation has completed.
- Custom application condition — a business state, e.g. a success alert appears.

PRECISION: Prefer explicit waits for specific state changes. Keep implicit wait at zero or a deliberately chosen minimal policy. Wait for business state — not arbitrary time.

KEY TAKEAWAY Wait for the customer row to appear, the success alert to show, or the submit button to become enabled — never for a fixed number of seconds.

WebDriver Architecture

WebDriver is the browser automation interface of Selenium. It acts as a bridge between test scripts and the browser, using the W3C WebDriver protocol over HTTP.



LOCATORS AND ELEMENT SELECTION

Locators tell Selenium which element on the page to interact with.

LOCATOR	EXAMPLE	WHEN TO USE
By.id	By.id("login-btn")	Stable when the app provides one — not inherently "fastest"
By.name	By.name("username")	Good for form fields
By.cssSelector	By.cssSelector(".product-card")	Flexible, readable, widely preferred
By.xpath	By.xpath("//button[text()='Buy']")	Avoid long/absolute paths; short, intention-revealing XPath can be reliable
By.linkText	By.linkText("Sign Out")	Breaks under localization or copy changes — semantic locators are safer
By.className	By.className("form-control")	Use when a class value is stable and unique enough
By.partialLinkText	By.partialLinkText("Forgot")	Same fragility as linkText under copy/text changes

LOCATOR PRIORITY — BY STABILITY, NOT SPEED

1. data-testid
2. Stable ID / semantic attr
3. Accessible role/label
4. CSS selector
5. XPath (last resort)

By.className: "form-control" is valid (single class token). "form-control active" is INVALID (compound string). For multiple classes use By.cssSelector(".form-control.active").

KEY TAKEAWAY Prefer locators by stability, not speed: data-testid first, then stable IDs/semantic attributes, then CSS — keep XPath as a last resort.

LOCATORS AND ELEMENT SELECTION

Locators are used to identify and select web elements on a page.

1. WHAT ARE LOCATORS?

Locators are strategies used by WebDriver to find web elements in the DOM.



Test Script

Locates

```
<div id="user">
  John Doe
</div>
```

Web Element

2. COMMON LOCATOR STRATEGIES

Locator	Description	Syntax Example	WebDriver (Java) Example	Best Used When
ID	Selects element by unique ID attribute.	#idValue	driver.findElement(By.id("userId"));	ID is unique and stable.
name	Selects element by the name attribute.	[name='nameValue']	driver.findElement(By.name("username"));	Name attribute is unique.
.class	Selects elements by class attribute.	.classValue	driver.findElement(By.className("btn-login"));	Use when class is unique.
<>	Selects elements by HTML tag name.	tagName	driver.findElement(By.tagName("input"));	Use for generic tags (e.g., input, button).
Link Text	Selects link by exact visible text.	linkText	driver.findElement(By.linkText("Forgot Password?"));	For links with exact text.
Partial Link Text	Selects link by partial visible text.	partialLinkText	driver.findElement(By.partialLinkText("Forgot"));	For links with dynamic text.
CSS	Uses CSS selector syntax.	cssSelector	driver.findElement(By.cssSelector("div.container > input[type='text']"));	Powerful and flexible.
XP	Uses XPath expression to find elements.	xpathExpression	driver.findElement(By.xpath("//input[@id='userId']"));	Use for complex conditions.

3. SAMPLE HTML SNIPPET

```
<form id="loginForm">
  <input type="text" id="userId"
    name="username" class="txt"
    placeholder="User ID" />
  <input type="password" id="pwd"
    name="password" class="txt"
    placeholder="Password" />
  <button type="submit" class="btn"
    id="loginBtn">Login</button>
  <a href="forgot.html" class="link"
    >Forgot Password?</a>
</form>
```

4. LOCATING ELEMENTS – EXAMPLES

Locator Strategy	Example (WebDriver – Java)
ID	driver.findElement(By.id("userId"));
name	driver.findElement(By.name("username"));
.class	driver.findElements(By.className("txt"));
<>	driver.findElements(By.tagName("input"));
Link Text	driver.findElement(By.linkText("Forgot Password?"));
Partial Link Text	driver.findElement(By.partialLinkText("Forgot"));
CSS	driver.findElement(By.cssSelector("#loginForm .btn"));
XP	driver.findElement(By.xpath("//input[@type='password']"));

5. ELEMENT SELECTION FLOW

Test Script

Choose Locator Strategy

Find Element(s)

Perform Actions

Validate Results

If Element Not Found

6. BEST PRACTICES

- ✓ Prefer ID over other locators.
- ✓ Use stable and unique attributes.
- ✓ Avoid absolute XPath.
- ✓ Use relative XPath or CSS for maintainability.
- ✓ Use meaningful attribute values for automation.
- ✓ Re-validate locators when UI changes.

7. LOCATOR PRIORITY (RECOMMENDED ORDER)

High (Preferred)

Low (Last Resort)

1 ID

2 Name

3 CSS Selector

4 Link Text / Partial Link Text

5 Class Name

6 Tag Name

7 XPath

KEY TAKEAWAY

Effective element location is critical for reliable and maintainable automation scripts. Choose the most stable and unique locator strategy for better test resilience.

© 2026 by Innovation In Software Corporation

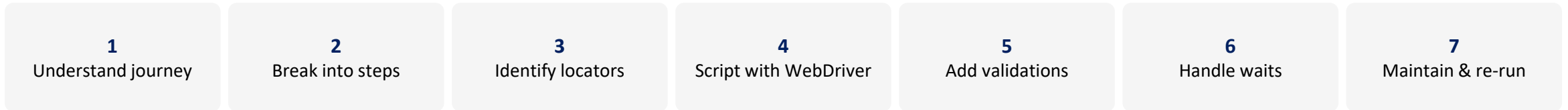
20

AUTOMATING A MAINTAINABLE USER JOURNEY

One realistic journey, a compact workflow, and locators that hold up as the UI changes.



COMPACT WORKFLOW — JOURNEY TO MAINTAINABLE TEST



SEE NEXT SLIDE: The Page Object Model — a CustomerFormPage example, assertion-ownership guidance, and when to stop abstracting.

KEY TAKEAWAY Automating a workflow means turning a real user journey into a repeatable, maintainable test script, validated end-to-end — not just recording clicks.

PAGE OBJECT MODEL

Separate locators and actions from scenario assertions — the pattern the lab's CustomerFormPage follows.

CustomerFormPage.java

```
public final class CustomerFormPage {
    private final WebDriver driver;
    private final WebDriverWait wait;
    private final By name =
        By.cssSelector("[data-testid='customer-name']");
    private final By email =
        By.cssSelector("[data-testid='customer-email']");
    private final By save =
        By.cssSelector("[data-testid='save-customer']");
    public CustomerFormPage enterCustomer(
        String customerName,
        String customerEmail) {
        driver.findElement(name).sendKeys(customerName);
        driver.findElement(email).sendKeys(customerEmail);
        return this;
    }
    public void save() {
        wait.until(
            ExpectedConditions.elementToBeClickable(save)
        ).click();
    }
}
```

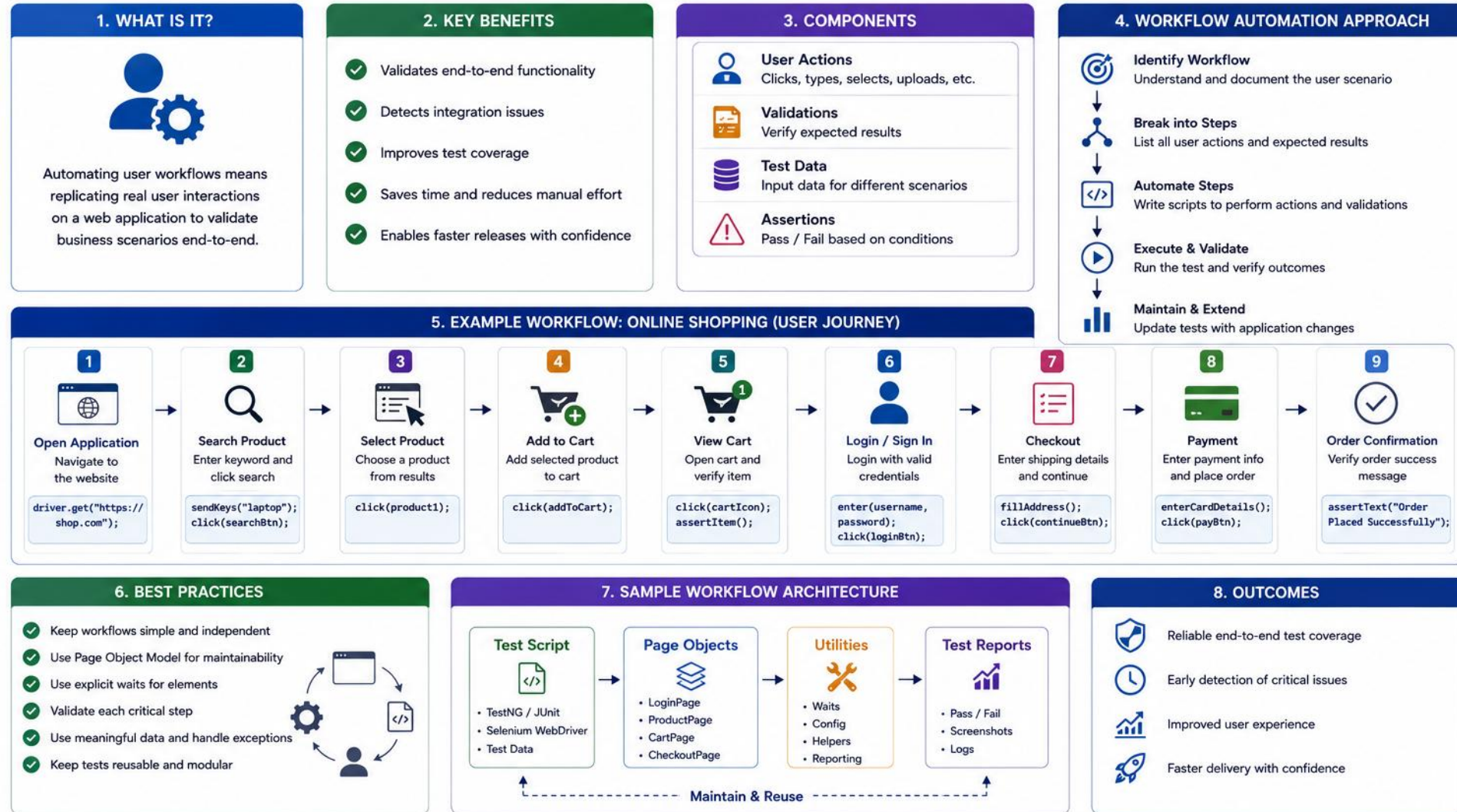
ASSERTION OWNERSHIP: The page object exposes page state and user actions; the test owns scenario-level assertions. Small guard assertions inside page objects may confirm page readiness — avoid hiding the whole test inside page methods.

DON'T OVER-ABSTRACT: Use page objects for stable screens or meaningful components; use component objects for reusable widgets; use workflow helpers sparingly. Avoid a separate page class for every tiny fragment.

KEY TAKEAWAY A page object exposes what the page can do; the test decides what "correct" means.

AUTOMATING USER WORKFLOWS

Automate real user scenarios by combining actions, validations and test data.

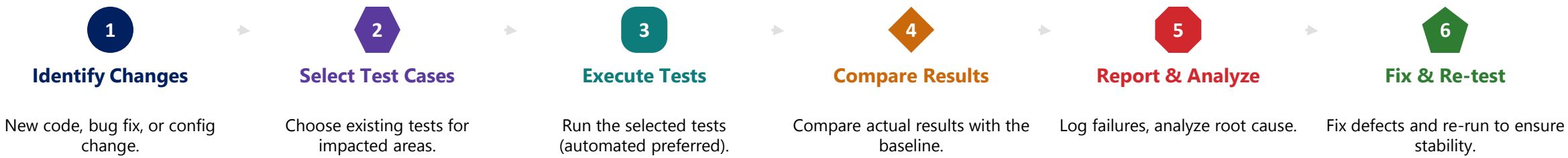


AUTOMATING USER WORKFLOWS = REAL USER JOURNEY + AUTOMATION + VALIDATION
Ensure your application works the way your users expect!

REGRESSION STRATEGY

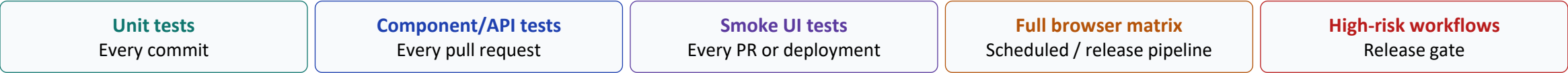
A repeatable workflow, corrected regression categories, and a cadence for running each kind of test.

REGRESSION TESTING WORKFLOW



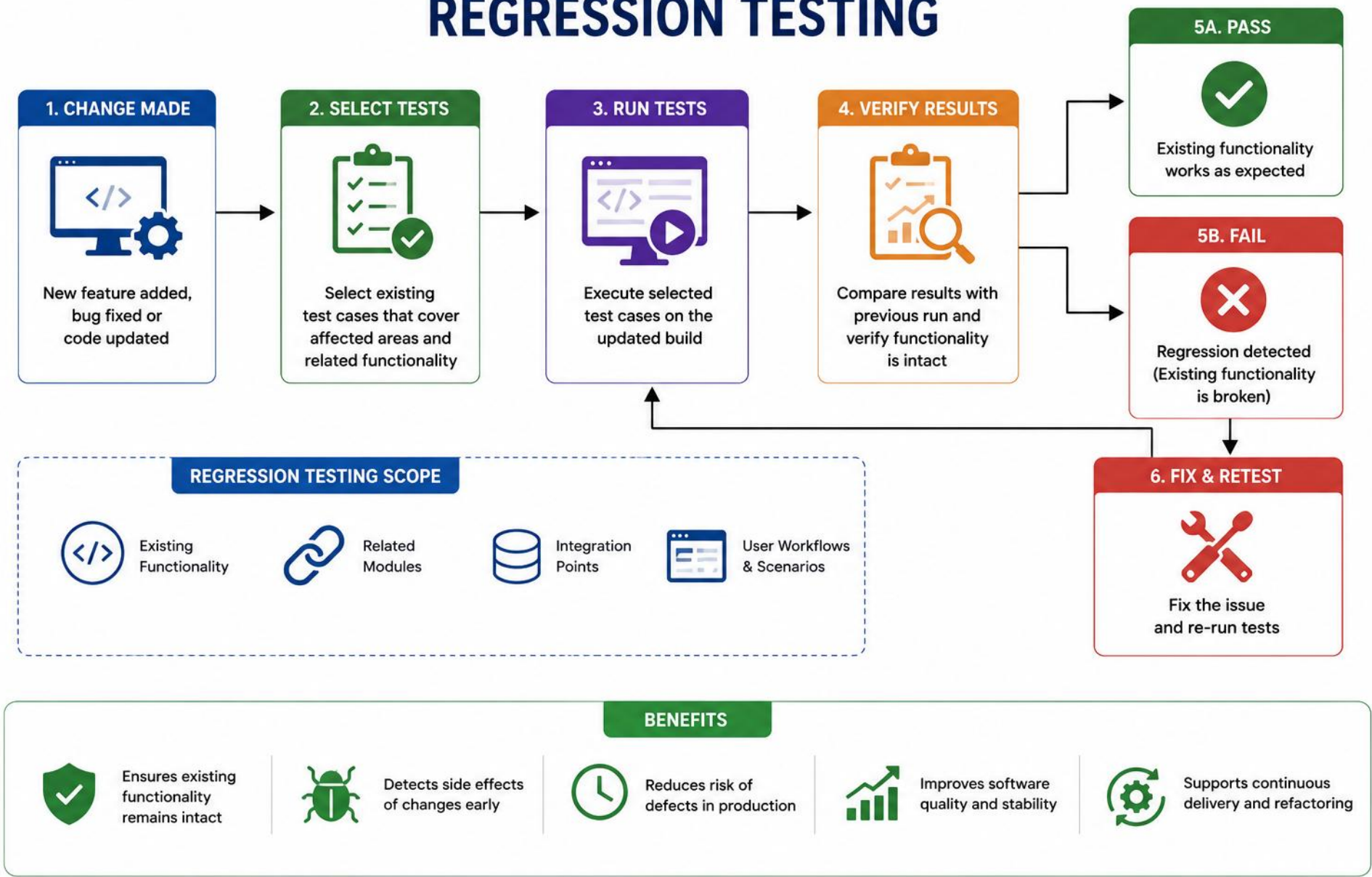
TYPE	DESCRIPTION
Smoke Regression	A small, fast subset confirming core functionality still works after any change.
Change-Impact (Selective)	Runs tests targeted at the areas a specific change could plausibly affect.
Full Regression	Executes the entire test suite to ensure the whole application is unaffected.
Exploratory Regression	Targeted manual exploration for new, changing, or highly visual behavior not yet covered by automation.

EXECUTION CADENCE



KEY TAKEAWAY Regression follows identify-select-execute-compare-fix, at smoke, change-impact, full, or exploratory scope — automate the stable scenarios and explore the rest by hand.

REGRESSION TESTING



TEST AUTOMATION BEST PRACTICES

Following these practices keeps your automation suite reliable, fast, and maintainable.

PRACTICE	WHY IT MATTERS	TOOL / TECHNIQUE
Page Object Model	Keeps locators and workflows maintainable	Page classes per screen
Explicit Waits	Avoids flaky tests from timing issues	WebDriverWait, expected conditions
Data-Driven Testing	Covers more scenarios with less code	JUnit 5 @ParameterizedTest, @CsvSource
Meaningful Assertions	Confirms outcomes, not just execution	Clear, specific assertions
Screenshots on Failure	Speeds up debugging failed tests	Automatic capture on assertion failure
CI/CD Integration	Fast feedback on every change	Surefire (*Test) + Failsafe (*IT) reports

MAVEN TEST EXECUTION (IMPORTANT): Classes named *Test run under Maven Surefire (unit tests); classes named *IT run under Maven Failsafe (integration tests) — the lab’s CustomerApiIT needs Failsafe, not Surefire. Use separate Maven profiles or pipeline stages (unit, integration, ui) since Selenium tests need browser availability, display/headless config, app server lifecycle, and ports — for clearer failure diagnosis.

PARALLEL EXECUTION: Run in parallel only after tests are independent and data-isolated: unique users/IDs, separate browser instances, isolated DB state, independent ports, thread-safe page objects, no static WebDriver.

Don'ts: don't automate everything, don't use Thread.sleep(), don't ignore failed tests. Deterministic fixtures are fine — avoid shared mutable data, environment-specific values, and colliding IDs.

KEY TAKEAWAY Reliable automation avoids fixed sleeps and brittle locators — explicit waits and the Page Object Model are essential.

LAB OVERVIEW – INTEGRATION AND UI TESTING WITH SELENIUM

Extend the CRM app with Spring HTTP integration tests and a Selenium Page Object UI suite.

APPLICATION UNDER TEST

- Northstar CRM (Customer Management Platform): create and get customer records, reusing the CUS-1001 / CUS-1002 fixtures and correlation ID from Labs 17–18.
- **Database strategy: this lab keeps the existing in-memory repository with reset hooks, for time-limited training. Use PostgreSQL Testcontainers (provided setup) when the goal is realistic persistence integration.**

LAB OVERVIEW

- You will extend lab18-crm with Spring Web and Selenium: write CustomerApiIT integration tests, build a Page Object UI suite for the CRM customer form, add negative test cases, and prove the regression suite stays green across two runs.
- This is a substantial implementation project. Starter code is provided for the Spring Boot application, controller, static HTML page, Maven plugins, browser configuration, and test base class, so you can focus on API assertions, the Page Object, wait strategy, positive/negative workflows, and evidence collection.

Prerequisites: Labs 17–18 (customer domain, service rules, Mockito tests), JDK 21, Maven, Chrome/Chromium, and Git basics.

TECH STACK

ITEM	VALUE
Language	Java, Maven
UI Automation	Selenium WebDriver, JUnit 5
API Testing	Spring Test, TestRestTemplate
Test Support	Maven Surefire, WebDriverManager

- **LAB ARCHITECTURE:**
API: CustomerApiIT → RANDOM_PORT → Controller → Service → Repository → Test DB

UI: CustomerUiIT → Headless Chrome → Running App → UI/API → Repository

KEY TAKEAWAY This lab combines Spring HTTP integration tests and a Selenium Page Object UI suite into one regression-minded workflow for the CRM.

LAB TASKS AND DELIVERABLES

Build HTTP integration tests and a Selenium UI suite to validate the CRM app's create/get flow end-to-end.

#	TASK	DESCRIPTION	TECH
1	Environment Setup	Copy lab18-crm to lab19-crm; add Spring Web, Selenium, WebDriverManager deps.	Java, Maven, Spring Boot
2	Create/Get API	Implement create/get endpoints with X-Correlation-Id echo.	Spring Web, Spring Boot
3	HTTP Integration Tests	Write CustomerApiIT: create, get, and 404 cases via HTTP.	Spring Test, TestRestTemplate
4	Minimal UI Surface	Build customers.html with data-testid hooks for the form.	HTML, Spring Boot statics
5	WebDriver Setup	Configure WebDriverManager and headless Chrome with waits.	Selenium, WebDriverManager
6	Page Object & UI Test	Build CustomerFormPage; automate Amina's happy-path create.	Selenium WebDriver, JUnit 5
7	Negative Test Cases	Add blank-name UI validation and a missing-customer 404 case.	Selenium, Spring Test
8	Regression & Evidence	Run suite twice; capture and restore a broken-locator screenshot.	Maven Failsafe, Selenium

VALIDATION, 404s, FAILURE EVIDENCE & RELIABILITY: Browser blocks empty submission; API test (task 3) checks Bean Validation; UI (task 7) shows the mapped error. 404 is tested as both an API and a UI case. On failure, capture: screenshot, page source, console logs, URL, network logs, server logs, correlation ID, test name, browser/version. Reliability: clean-checkout pass, pass alone and as a suite, reset state, unique data, no sleep(), no order dependence.

MAVEN LIFECYCLE, CORRELATION ID & LOCATOR DEMO: CustomerServiceTest → Surefire (mvn test); CustomerApiIT/CustomerUiIT → Failsafe (mvn verify). Define Spring Boot startup, the random port reaching Selenium, browser start/stop, screenshot capture, and data reset. X-Correlation-Id: validate length/characters, generate if absent, never use for auth, block log injection. Broken-locator demo is an OPTIONAL instructor exercise, not mandatory learner work.

KEY TAKEAWAY CustomerApiIT and CustomerUiIT both pass; negative cases are covered; the regression suite stays green across two runs.

INTEGRATION AND UI TEST DEFINITION OF DONE

A checklist for a genuinely complete integration and UI test suite — not just a restatement of the objectives.

- ✓ Test boundary is explicitly defined
- ✓ Database technology matches the risk being tested
- ✓ Tests assert business outcomes, not only clicks
- ✓ Real components under test are identified
- ✓ HTTP tests run against a clear web environment
- ✓ Browsers and server resources are always cleaned up
- ✓ Unrelated external systems are controlled
- ✓ Browser tests use stable test hooks
- ✓ Failure evidence is captured
- ✓ Test data is deterministic and isolated
- ✓ Explicit waits replace fixed sleeps
- ✓ Suites are separated by speed and purpose
- ✓ Production data is never used
- ✓ Page objects separate locators from scenarios
- ✓ Tests run reliably from Maven and CI

KEY TAKEAWAY Fifteen checks, not ten objectives — that's what separates a demo from a suite you can trust in CI.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of integration testing and UI test automation.

1. A repository test passes with H2 but fails against PostgreSQL due to database-specific SQL. What should change?

- A. Ignore it — H2 is close enough
- B. Use an integration test against PostgreSQL (e.g. Testcontainers)**
- C. Remove the test since databases are unpredictable
- D. Rewrite the repository to avoid SQL entirely

3. What is Selenium WebDriver's role?

- A. It stores test results in a database
- B. It sends commands to the browser driver**
- C. It replaces JUnit for assertions
- D. It only tests REST APIs

2. What is the main goal of integration testing?

- A. Test a single method in isolation
- B. Verify components work correctly together**
- C. Replace unit tests entirely
- D. Test only the UI

4. Which locator is generally most maintainable when the app provides a stable, dedicated test hook?

- A. By.xpath with an absolute path
- B. By.linkText
- C. By.id, even if auto-generated and unstable
- D. A selector based on a stable data-testid**

KEY TAKEAWAY Integration tests verify real component wiring; Selenium WebDriver automates real browser interactions.

KNOWLEDGE CHECK (2 OF 2)

Four more questions, plus the full answer key.

5. A Selenium test uses `Thread.sleep(5000)` after every click. What is the better approach?

- A. Increase the sleep to 10000ms to be safe
- B. Wait for the specific expected UI state using an explicit wait**
- C. Remove all waits entirely
- D. Use `Thread.sleep()` only in CI, not locally

7. What is a key benefit of the Page Object Model?

- A. It hides bugs from testers
- B. It keeps locators and workflows maintainable**
- C. It removes the need for assertions
- D. It only works with REST Assured

ANSWER KEY

• 1-B 2-B 3-B 4-D 5-B 6-B 7-B 8-C

6. What is the purpose of regression testing?

- A. To test a feature for the first time
- B. To catch regressions after code changes**
- C. To replace manual testing entirely
- D. To test only the database layer

8. Where do end-to-end tests sit in the test pyramid?

- A. At the base — many, fast tests
- B. In the middle — some, moderate-speed tests
- C. At the top — few, slow, high-value tests**
- D. They are not part of the test pyramid

KEY TAKEAWAY Reliable, maintainable automation — explicit waits, stable locators, and the Page Object Model — is what makes CI/CD safe.

*"Good automation is reliable, maintainable,
and provides confidence in every release."*

MODULE 19 COMPLETE

Integration Testing and UI Test Automation

You can now validate applications end-to-end — from integrated components to automated real user workflows.